

Resumen teórico — Sistemas Operativos

Capítulo 4 de las Notas del curso: *El lenguaje C y herramientas*

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento es un resumen teórico del capítulo 4 de las *Notas del curso* de Sistemas Operativos (Marcelo Arroyo, UNRC), dedicado al lenguaje C y al *toolchain* de desarrollo: pre-procesamiento, compilación, ensamblado, linking, archivos objeto, ejecutables, símbolos, bibliotecas estáticas y dinámicas, carga dinámica explícita y sistemas de *build* como GNU make. El objetivo es servir como material de estudio: las definiciones formales se acompañan de explicaciones intuitivas, ejemplos y conexiones entre conceptos, de modo que el lector pueda preparar tanto el examen teórico como el Práctico 2 con una comprensión sólida y articulada del proceso que transforma un archivo `.c` en un ejecutable.

Índice

1. Introducción	2
2. El lenguaje C	2
2.1. Tipos de datos	2
2.2. Punteros	3
2.3. Funciones, definiciones y declaraciones	3
2.4. Módulos: archivos <code>.c</code> y <code>.h</code>	4
3. El proceso de compilación	4
3.1. Cuatro etapas	4
3.2. Detener el proceso en una etapa	4
3.3. El rol del pre-procesador	5
4. Archivos objeto	5
4.1. Definición y formato	5
4.2. Secciones típicas	5
4.3. Símbolos: definidos y no definidos	6
4.4. Tabla de reubicación	6
4.5. Direcciones en un objeto relocatable	6
5. Enlazado (linking)	6
5.1. Función del linker	6
5.2. Resolución estática	7
5.3. Resolución dinámica (introducción)	7
5.4. Ejemplo completo: tres llamadas	7
6. Bibliotecas	7
6.1. Bibliotecas estáticas	8
6.2. Bibliotecas dinámicas	8
6.3. Lazy linking	9
6.4. Carga dinámica explícita: <code>dlopen/dlsym</code>	9

7. Espacio de direcciones del ejecutable	9
8. Build systems: make y los Makefiles	10
8.1. Motivación	10
8.2. GNU make	10
8.3. Variables automáticas y reglas de patrón	11
8.4. Compilación incremental	11
8.5. Autotools y otros sistemas de build	11
9. Resumen integrador	12
Referencias	12

1. Introducción

Programar en C parece resumirse a un único comando —`gcc -o myprog main.c hello.c`—, pero detrás de él hay una secuencia de herramientas trabajando en conjunto. Entender qué hace cada una es clave porque el sistema operativo *ejecuta* ese resultado: el formato del binario, la disposición del código y los datos en memoria, y la forma en que las funciones de biblioteca se enlazan en tiempo de ejecución dependen directamente de las decisiones que se toman en cada etapa del proceso de compilación.

Definición

La **cadena de herramientas** (*toolchain*) es el conjunto de programas que transforman un programa fuente en un ejecutable. Incluye, como mínimo, un *pre-procesador*, un *compilador*, un *ensamblador* y un *linker*.

Idea intuitiva

Pensemos en la analogía con una receta: el código fuente es la receta escrita en lenguaje humano, el assembly es la lista de pasos detallados, el archivo objeto es el plato hecho pero todavía sin montar, y el ejecutable es la comida lista para servir. El *linker* es el que arma el plato uniendo los ingredientes (funciones) que provienen de varios módulos.

2. El lenguaje C

C es un lenguaje de bajo nivel, diseñado por Dennis Ritchie en los años 70, con abstracciones simples y directas al hardware. UNIX fue el primer sistema operativo escrito casi enteramente en C, y desde entonces el lenguaje quedó indisolublemente ligado a la programación de sistemas: drivers, bibliotecas estándar, kernels, intérpretes y compiladores se escriben típicamente en C.

2.1. Tipos de datos

Básicos.

- **Enteros:** `char` (un byte), `short int`, `int`, `long int`. Pueden ser **signed** (con signo) o **unsigned** (sin signo). Los literales se escriben en decimal (456), octal (0367) o hexadecimal (0x20AF).
- **Punto flotante:** `float` (simple precisión) y `double` (doble precisión).
- **Enumeraciones:** dan nombres simbólicos a constantes enteras consecutivas.

Compuestos.

- **Arreglos:** elementos contiguos en memoria, indexados desde 0 hasta N-1. El nombre del arreglo, en la mayoría de los contextos, decae a un puntero a su primer elemento.
- **Strings:** arreglos de caracteres *terminados en \0* (NULL). El literal "hola" se almacena como {'h','o','l','a','\0'}.
- **Estructuras (struct):** agrupan campos heterogéneos bajo un mismo nombre. Cada campo se accede con . o, si se tiene un puntero, con ->.
- **Uniones (union):** campos de tipos distintos que comparten la misma posición de memoria. Sirven para implementar *polimorfismo* de bajo nivel (representaciones alternativas del mismo dato).

Constantes y alias. Los valores constantes se declaran anteponiendo la palabra clave `const` al tipo. Los alias de tipo se introducen con `typedef`, lo que permite escribir, por ejemplo, `typedef unsigned char byte;` y luego usar `byte` como sinónimo de `unsigned char`.

2.2. Punteros

Definición

Un **puntero** es una variable cuyo valor es la dirección de memoria de otra variable. Un puntero a un valor de tipo T se declara como `T *p`. Tiene asociados dos operadores: `&v` (tomar la dirección de v) y `*p` (desreferenciar p, es decir, leer el valor apuntado).

```
1 int v = 10;
2 int *ptr = &v; /* ptr apunta a v */
3 printf("v = %d\n", *ptr); /* desreferencia: imprime 10 */
```

En la declaración `T v[N]`, el identificador `v` es un *puntero constante* a su primer elemento; por lo tanto, `*v == v[0]`.

Aritmética de punteros. La aritmética de punteros tiene en cuenta el tamaño del tipo apuntado. `ptr + 1` no significa "sumar un byte" sino "avanzar al siguiente elemento del arreglo", es decir, `ptr + sizeof(*ptr)` en bytes.

Punteros a función. C permite tomar la dirección de una función y guardarla en un puntero. Esto habilita *programación de alto orden*: pasar funciones como parámetros, instalar callbacks, despachar dinámicamente. La sintaxis es incómoda pero idiomática:

```
1 typedef int (*fptr_t)(int, int); /* puntero a función int x int -> int */
2 int inc(int v, int step) { return v + step; }
3 fptr_t f = inc;
4 printf("%d\n", f(1, 2)); /* imprime 3 */
```

2.3. Funciones, definiciones y declaraciones

C distingue dos conceptos relacionados pero distintos:

Definición

La **definición** de una función incluye su *cuerpo*: el bloque de sentencias que ejecuta. Solo puede haber una definición por función en todo el programa.

Definición

Una **declaración** (también llamada *prototipo*) especifica únicamente el *perfil* de la función —tipo de retorno, nombre y tipos de los parámetros— pero no su cuerpo. Sirve para que el compilador valide el uso cuando la definición está en otro lugar (más abajo en el archivo o en otro módulo).

Los parámetros se pasan *por valor*; el pasaje por referencia se *simula* pasando un puntero. C admite además *funciones variádicas* (con un número arbitrario de argumentos), como la propia `printf`, cuyo perfil es `int printf(const char *fmt, ...)`.

2.4. Módulos: archivos .c y .h

C no tiene una construcción sintáctica para módulos. Por convención cada módulo se distribuye en dos archivos:

- Un archivo de implementación `nombre.c` con las *definiciones* (cuerpo de funciones, variables globales).
- Un archivo de cabecera `nombre.h` con las *declaraciones* de los símbolos exportados, pensado para ser incluido (`#include "nombre.h"`) por otros módulos que lo usen.

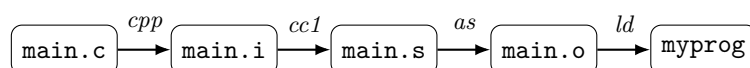
Las cabeceras del sistema se incluyen con `<>` (por ejemplo, `#include <stdio.h>`); las del proyecto, con comillas (`#include "miarchivo.h"`). El compilador busca cada cabecera en distintos directorios según el caso.

3. El proceso de compilación

3.1. Cuatro etapas

Compilar un programa C consiste de cuatro pasos coordinados por el *driver gcc* (o *clang*):

1. **Pre-procesamiento (cpp)**. Procesa las directivas del programa fuente y *expande* las macros y `#include`. Su salida es un archivo temporal con extensión `.i`.
2. **Compilación (cc1)**. Traduce el código C ya expandido a *assembly* de la arquitectura objetivo. Salida: `.s`.
3. **Ensamblado (as)**. Convierte el assembly en un *archivo objeto binario* en el formato del SO (ELF en GNU/Linux, Mach-O en macOS, PE/COFF en Windows). Salida: `.o`.
4. **Linking (ld)**. Combina archivos objeto y bibliotecas, recalcula direcciones y resuelve referencias externas para producir el *ejecutable* (o una nueva biblioteca).



3.2. Detener el proceso en una etapa

`gcc` permite frenar la cadena de compilación en cualquier etapa intermedia, lo que es muy útil para inspección y depuración:

```
gcc -E hello.c -o hello.i # solo preprocesa
gcc -S hello.c -o hello.s # preprocesa + compila
gcc -c hello.c -o hello.o # preprocesa + compila + ensambla
gcc hello.c -o hello # ciclo completo
```

Flag	Detiene tras	Salida
-E	Pre-procesamiento.	Archivo <code>.i</code> (texto).
-S	Compilación.	Archivo <code>.s</code> (assembly).
-c	Ensamblado.	Archivo <code>.o</code> (binario).
(sin flags)	Linking.	Ejecutable.

Cuadro 1: Opciones de gcc para detener la compilación.

3.3. El rol del pre-procesador

El pre-procesador no es un compilador: es un manipulador de texto. Sus directivas más comunes son:

- **#include**: copia el contenido de otro archivo.
- **#define**: define una macro de sustitución textual.
- **#ifdef/#endif**: compilación condicional.

Por ejemplo, dado `#define hi "Hello world"` y un cuerpo `return hi;`, el pre-procesador entrega al compilador `return "Hello world";`: la macro `hi` no llega siquiera a la etapa de compilación, así que no aparece en la tabla de símbolos del objeto.

Idea intuitiva

La regla mental útil es: *si está con #, lo resuelve el preprocesador antes de mirar el código C*. Esa es la razón por la que las macros no respetan tipos ni alcance de bloques: son sustitución de texto pura.

4. Archivos objeto

4.1. Definición y formato

Definición

Un **archivo objeto** (`.o`) es un binario en el formato definido por la *ABI* del sistema. Contiene el código y los datos de un módulo en una representación que el linker puede combinar con otros objetos para producir un ejecutable o una biblioteca.

Cada plataforma usa un formato distinto:

- **ELF** (*Executable and Linkable Format*) en GNU/Linux y muchos UNIX modernos.
- **Mach-O** en macOS y otras plataformas Apple.
- **PE/COFF** (*Portable Executable / Common Object File Format*) en Windows.

Los detalles cambian, pero la idea conceptual es la misma: un *header* con metadatos seguido de un conjunto de *secciones* con propósitos bien definidos.

4.2. Secciones típicas

- **.text**: instrucciones del programa (código ejecutable, solo lectura).
- **.data**: variables globales y estáticas con valores iniciales no nulos. Se subdivide en datos *inicializados* y *no inicializados* (esto último a veces aparece como `.bss`).
- **.rodata** (en macOS: `__TEXT, __cstring`): constantes y literales de cadena, de solo lectura.
- **Symbol table**: mapping *símbolo* → *dirección* que el linker usa para resolver referencias.

- **Reallocation entries:** lista de instrucciones cuyos operandos hacen referencia a símbolos externos y necesitan ser *parchadas* por el linker.

4.3. Símbolos: definidos y no definidos

Definición

Un **símbolo** es un identificador (función, variable global, constante) con una dirección dentro del archivo objeto. Si la dirección está asignada, el símbolo está *definido* en ese módulo; si solo aparece como referencia sin definición, está *indefinido* (**UND**) y el linker deberá resolverlo buscando el módulo o la biblioteca que lo defina.

La tabla de símbolos es un *mapping* de la forma $\text{symtbl}(\text{symbol}) \rightarrow \text{address}$. Por ejemplo, en un archivo `main.o` que llama a `printf` y a `hello`, ambos símbolos aparecen como **UND** porque sus definiciones viven en otros módulos: `libc` y `hello.o`, respectivamente.

4.4. Tabla de reubicación

Definición

La **tabla de reubicación** (*relocation table*) es un mapping de la forma $\text{relloc}(\text{instr_address}, \text{instr_type}) \rightarrow \text{external_symbol}$ que indica al linker qué operandos de qué instrucciones debe completar al combinar los objetos.

Las entradas de la tabla de reubicación corresponden, típicamente, a instrucciones de la forma `call <addr>`, `load/store <addr>`, etc., donde `<addr>` todavía no se conoce porque el símbolo es externo. El compilador deja 0 (o un valor de relleno) en el operando y agrega la entrada correspondiente.

4.5. Direcciones en un objeto relocatable

En un archivo objeto cada función empieza, típicamente, en el *offset* 0x0 de su sección `.text`: cada módulo "se siente solo en el mundo". Las direcciones definitivas las asigna el linker cuando combina varios objetos en un mismo binario.

5. Enlazado (linking)

5.1. Función del linker

Definición

El **linker** (también *link-editor*) es la herramienta encargada de producir binarios (ejecutables o bibliotecas) a partir de la *composición* de varios archivos objeto y bibliotecas.

Su trabajo se descompone, conceptualmente, en dos pasos:

1. **Concatenar** las secciones de código y datos de cada archivo objeto y *recalcular* las direcciones de variables, constantes y funciones. Cada `.text` que empezaba en offset 0 pasa a ocupar una dirección absoluta en el ejecutable.
2. **Resolver las referencias externas:** completar los operandos de las instrucciones de llamada y de los accesos a variables globales externas, usando para ello las tablas de símbolos y de reubicación de cada objeto.

5.2. Resolución estática

Definición

El **linking estático** es la resolución de referencias externas en *tiempo de compilación*: el linker copia el código necesario dentro del ejecutable final y completa todas las direcciones.

Si el módulo `main.o` llama a `hello` (definido en `hello.o`), el linker, al combinar ambos, conoce la dirección final de `_hello` y completa el operando del `call` con esa dirección. La invocación queda *resuelta* y el ejecutable es *auto-contenido* respecto de `hello`.

5.3. Resolución dinámica (introducción)

Definición

El **linking dinámico** es la resolución de referencias externas en *tiempo de carga* o de *primera invocación*: el código de la biblioteca no se copia al ejecutable, sino que se mantiene en una biblioteca compartida que el SO carga aparte.

Las funciones de la libc, como `printf`, viven en una biblioteca compartida (`libc.so`, `libSystem.B.dylib`). El linker estático no copia el código de `printf` al ejecutable: solo deja una referencia que será resuelta por el *dynamic linker* cuando el programa se cargue. Más adelante se describe el mecanismo en detalle.

5.4. Ejemplo completo: tres llamadas

Ejemplificando con los programas `main.c` y `hello.c` de las notas: `main.o` contiene dos referencias externas (`_hello` y `_printf`). Antes del linking, el disassembly muestra ambos `call` apuntando a 0 (sin resolver):

```
.text section (main.o)
_main:
0x00 ...
0x18 call 0x18 ; <-- _hello, sin resolver
0x1c call 0x1c ; <-- _printf, sin resolver
```

Luego del linking, en el ejecutable `myprog`:

```
.text section (myprog)
main:
0x... call hello ; resuelto estaticamente
0x... call printf_stub ; resolver dyld
hello:
0x... ...
    ret
```

El `call` a `hello` apunta directamente al inicio de la función `_hello` dentro del ejecutable. El `call` a `printf` apunta a un *stub* (más abajo).

6. Bibliotecas

Definición

Una **biblioteca** es una colección de módulos (código y datos) *reusables*. Generalmente contiene un conjunto de archivos objeto. Pueden ser de dos tipos: para *enlazado estático* o de *enlazado dinámico* (*shared*).

6.1. Bibliotecas estáticas

Definición

Una **biblioteca estática** es un *contenedor* (formato *archiver* en sistemas tipo UNIX) de archivos objeto. Incluye un *índice* con los nombres de los símbolos exportados por cada miembro.

En sistemas tipo UNIX se generan con la herramienta **ar**:

```
ar rcs libmilib.a file1.o file2.o ...
```

Las opciones significan: **r** reemplaza miembros existentes, **c** crea el archivo si no existe y **s** construye o actualiza el índice de símbolos. Por convención, una biblioteca **B** se nombra **libB.a**.

Cuando el linker busca un símbolo no resuelto, consulta el índice de las bibliotecas estáticas pasadas en la línea de comandos. Si lo encuentra, copia *solo el archivo objeto necesario* dentro del ejecutable.

```
gcc -o myprog -L. -lhello main.o
```

El flag **-L\$DIR** agrega **\$DIR** al *path* de búsqueda; **-lhello** le dice al linker que busque **libhello.a** (o, si está disponible, su versión dinámica).

6.2. Bibliotecas dinámicas

Definición

Una **biblioteca dinámica** (también *shared library*) es un objeto compartible que el SO carga y enlaza bajo demanda en *tiempo de ejecución*. Se nombra **libfoo.so** en GNU/Linux, **libfoo.dylib** en macOS y **foo.dll** en Windows.

Generación. Con **gcc** se generan en dos pasos: compilar a objeto con código *independiente de posición* (**-fPIC**) y luego linkear con la opción **-shared**.

```
gcc -c -fPIC hello.c # genera hello.o relocatable
gcc -shared -o libhello.so hello.o
```

Ventajas y desventajas.

- **Ventaja:** el ejecutable no incluye el código de la biblioteca; varios procesos pueden compartir una sola copia cargada en memoria. Las actualizaciones de la biblioteca benefician a todos los programas sin recompilar.
- **Desventaja:** sobrecarga al inicio del proceso (resolución de símbolos) y dependencia explícita: si la biblioteca no está, el programa no arranca. Se mitiga con *lazy linking*.

Preferencia del linker. Si conviven una versión estática y una dinámica de la misma biblioteca, el linker *prefiere la dinámica* salvo que se le indique explícitamente lo contrario (por ejemplo, pasando **libhello.a** directamente).

6.3. Lazy linking

Definición

Lazy linking es la técnica por la cual las funciones de una biblioteca dinámica no se resuelven todas al cargar el programa, sino *cada una en su primera invocación*. Reduce la latencia inicial del proceso a costo de un pequeño salto extra en cada llamada.

Mecanismo: stubs y GOT. El linker estático genera, para cada función externa, un pequeño fragmento de código llamado *stub* (ej. `printf_stub`) y una entrada en una tabla indirecta llamada **Global Offset Table (GOT)**. Esquemáticamente:

1. En el ejecutable, `call printf` apunta al stub.
2. El stub realiza un `jmp` indirecto leyendo la dirección de `GOT[printf]`.
3. Inicialmente, `GOT[printf]` apunta a una rutina de resolución del *dynamic linker*.
4. En la primera invocación, el dynamic linker localiza `printf` en `libc.so`, escribe su dirección verdadera en `GOT[printf]` y salta a ella.
5. Las invocaciones siguientes ya encuentran la dirección correcta en la GOT y saltan directamente, sin volver a invocar al dynamic linker.

Idea intuitiva

Los *stubs* usan `jmp` y no `call` porque deben retornar al programa que invocó originalmente, no al stub. Es la razón por la que la "indirección" no agrega un nivel innecesario en la pila.

6.4. Carga dinámica explícita: `dlopen/dlsym`

Hasta ahora se asume que las dependencias dinámicas se declaran en tiempo de linking (`-lhello`). Existe otra opción: la **carga dinámica explícita** mediante la API POSIX `<dlfcn.h>`, que permite a un programa cargar una biblioteca *en tiempo de ejecución* sin haberla declarado al compilar.

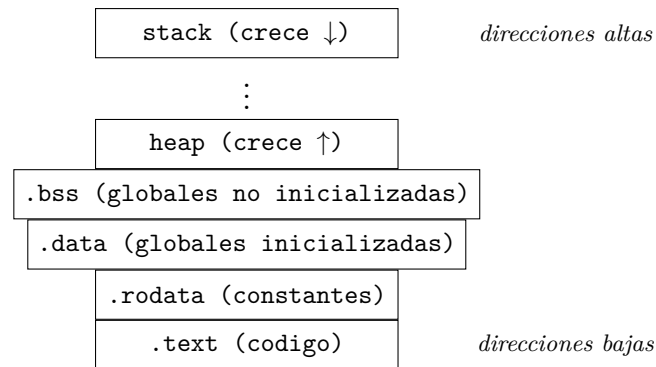
- `dlopen("ruta", flags)`: carga la biblioteca y devuelve un *handle* opaco. Los flags más usados son `RTLD_NOW` (resolver todos los símbolos al cargar) y `RTLD_LAZY` (resolverlos bajo demanda).
- `dlsym(handle, "nombre")`: resuelve el símbolo dentro de la biblioteca cargada y devuelve su dirección.
- `dlclose(handle)`: descarga la biblioteca.
- `dLError()`: devuelve el último mensaje de error y limpia el estado interno.

```
1 #include <dlfcn.h>
2
3 void *handle = dlopen("./libhello.so", RTLD_NOW);
4 char *(*fn)(void) = dlsym(handle, "hello");
5 printf("%s\n", fn());
6 dlclose(handle);
```

Esta técnica es la base de los sistemas de *plugins* y de cualquier mecanismo de extensibilidad en tiempo de ejecución (módulos cargables, backends seleccionables por configuración, etc.).

7. Espacio de direcciones del ejecutable

Cuando el SO carga un ejecutable, el espacio de direcciones del proceso queda dispuesto, conceptualmente, así (de direcciones bajas a altas):



- **Texto (.text)**: instrucciones del programa. De solo lectura, compartible entre procesos.
- **Datos inicializados (.data)**: variables globales y estáticas con valor inicial no nulo.
- **Datos no inicializados (.bss)**: variables globales y estáticas sin valor inicial; el kernel las inicia a cero.
- **Heap**: memoria dinámica obtenida con `malloc/brk`; crece hacia direcciones altas.
- **Stack**: pila de llamadas; crece hacia direcciones bajas. Cada invocación a una función crea un *stack frame* con sus parámetros, variables locales, dirección de retorno y registros guardados.

Idea intuitiva

Una variable local `int x;` dentro de una función vive en el *stack frame* de esa función y desaparece cuando la función retorna. Una variable global `int g = 42;` vive en `.data` durante toda la ejecución del programa. Una constante literal como `"hola"` vive en `.rodata` y es de solo lectura: intentar modificarla suele provocar `SIGSEGV`.

8. Build systems: make y los Makefiles

8.1. Motivación

En un proyecto real, las cuatro etapas del toolchain se ejecutan sobre decenas o cientos de archivos. Recompilar todo desde cero ante cada cambio es inviable. Un *build system* automatiza el proceso y, sobre todo, evita el trabajo redundante: solo reconstruye lo que realmente cambió.

8.2. GNU make

Definición

make es una herramienta que toma una especificación de *targets* y *dependencias* (escrita en un archivo **Makefile**) y construye automáticamente el grafo de dependencias. Comparando los *timestamps* de los archivos, decide qué reglas disparar y en qué orden.

La forma básica de una regla es:

```
target: dependencias
<TAB>comando
```

Importante: la indentación debe ser exactamente un TAB, no espacios.

Ejemplo idiomático para construir `myprog` a partir de `main.c` y `hello.c`:

```
myprog: main.o hello.o
    gcc -o myprog main.o hello.o
```

```
main.o: main.c
    gcc -c main.c

hello.o: hello.c
    gcc -c hello.c
```

8.3. Variables automáticas y reglas de patrón

Para no repetir información, **make** ofrece *variables automáticas*:

- `$@`: nombre del target.
- `$^`: lista completa de dependencias.
- `$<`: primera dependencia (la "fuente").

Y *reglas de patrón* que generalizan la construcción:

```
myprog: main.o hello.o
    gcc -o $@ $^

%.o: %.c
    gcc -c $<
```

8.4. Compilación incremental

make construye un grafo de dependencias y, en cada invocación, decide:

- Si el target no existe o es más viejo que alguna dependencia, hay que reconstruirlo.
- Antes de hacerlo, hay que asegurarse de que las dependencias estén actualizadas (recursión).

Idea intuitiva

Si `main.c` no cambió, **make** no recompila `main.o`, aunque sí pueda regenerar `hello.o` y relinkar el ejecutable. Es la diferencia entre un proyecto manejable y uno inviable.

8.5. Autotools y otros sistemas de build

Sobre **make** se construye el ecosistema de las **autotools**:

- **Autoconf**: genera el script `configure` a partir de un archivo `configure.ac`; el script detecta el entorno (compilador disponible, opciones) y produce un `Makefile` adaptado.
- **Automake**: a partir de `Makefile.am` y `configure.ac` genera el `Makefile.in` portable que usará `configure`.

El flujo típico para un usuario que recibe el *tarball* de un proyecto es:

```
tar xzvf my-package.tgz
cd my-package
./configure && make && sudo make install
```

Otros sistemas modernos (CMake, Meson, Ninja, Bazel) parten de las mismas ideas y agregan portabilidad, paralelismo o reproducibilidad.

9. Resumen integrador

Conviene cerrar visualizando todo el flujo en un único ejemplo. Sea el comando

```
gcc -o myprog main.c hello.c
```

Lo que ocurre detrás es lo siguiente:

1. `gcc` invoca al *pre-procesador* sobre cada `.c`: expande `#include`, resuelve `#define` y produce un `.i` temporal.
2. Pasa el `.i` al *compilador* (`cc1`), que lo traduce a *assembly* específico de la arquitectura (`.s`).
3. Invoca al *ensamblador* (`as`) sobre cada `.s` y obtiene los archivos objeto `main.o` y `hello.o`, en formato ELF/Mach-O. En cada uno, las funciones empiezan en offset `0x0` y las llamadas a símbolos externos quedan sin resolver, registradas en la tabla de reubicación.
4. Llama al *linker* (`ld`), que combina los objetos: concatena las secciones, recalcula direcciones, resuelve estáticamente `_hello` (al estar definido en `hello.o`) y deja `_printf` pendiente, con un *stub* y una entrada en la GOT, para que lo resuelva el *dynamic linker* en ejecución.
5. El binario resultante (`myprog`) tiene una sección `.interp` que indica el dynamic linker que debe cargarlo (`/lib/ld-linux.so` en Linux, `/usr/lib/dyld` en macOS) y una sección `.dynamic` con la lista de bibliotecas dinámicas que necesita (la `libc`, principalmente).
6. Al ejecutar `./myprog`, el SO carga el ejecutable, invoca al dynamic linker, este carga `libc` (si no está ya en memoria), inicializa la GOT y, finalmente, el kernel salta a `_start` (que termina invocando a `main`).
7. Cuando `main` llama por primera vez a `printf`, el stub salta vía la GOT al dynamic linker, que resuelve la dirección, la escribe en la GOT y salta a la verdadera `printf` de la `libc`. La siguiente invocación ya va directa.

Detrás de un comando aparentemente simple operan, en conjunto: el *pre-procesador*, el *compilador*, el *ensamblador*, el *linker* estático, el *loader* del SO, el *dynamic linker*, las *bibliotecas compartidas* y la *tabla de reubicación* de cada objeto. El estudio de la compilación y el linking consiste, precisamente, en entender cómo todas esas piezas funcionan juntas para transformar texto en código en ejecución.

Referencias

1. Marcelo Arroyo. *Notas del curso de Sistemas Operativos*, capítulo *El lenguaje C y herramientas*. UNRC, 2026.
<https://marceloarroyo.gitlab.io/cursos/SO/index.html#/toolchain>
2. Brian Kernighan, Dennis Ritchie. *The C programming language*, 2.^a edición. Prentice Hall, 1988.
3. John R. Levine. *Linkers and Loaders*. Morgan Kaufmann, 2000.
4. TIS Committee. *Tool Interface Standard (TIS) Executable and Linking Format (ELF) Specification*, v1.2.
5. POSIX.1-2017 / Single UNIX Specification. The Open Group, 2018.
6. Páginas de manual: `man 1 gcc`, `man 1 ld`, `man 1 ar`, `man 1 objdump`, `man 1 nm`, `man 3 dlopen`, `man 1 make`.